ALGORITHM DESIGN & STRATEGY ENGINE DOCUMENTATION

Premium JavaFX Blackjack Simulator

Portfolio-Grade Engineering Whitepaper

A full-stack Java desktop blackjack engine built with a reusable rules core, a JavaFX presentation layer, rule-based strategy analysis, animated table UX, persistent sessions, and recommendation analytics.

Java + JavaFX + FXML + CSS

Single-player vs Dealer

Strategy Analysis

Session Persistence

Recommendation Accuracy

Author: Hyeonjo Kim
Document Type: Software Engineering Design Document / Technical Whitepaper

Document Scope

- Core object model and engine responsibilities
- Round lifecycle and branching algorithms
- Rule-based strategy recommendation system
- Expected value, bust risk, and dealer bust chance heuristics
- JavaFX synchronization, animation, and state management
- Performance, persistence, and roadmap decisions

02. PROJECT OVERVIEW

Why This Engine Exists

This blackjack engine solves a common gap in game demos: most portfolio blackjack projects either stop at simple console play or bury the rules inside UI code. This project instead separates the core game engine, statistical analytics, strategy recommendation logic, and JavaFX presentation into distinct layers.

The result is not just a playable casino-style desktop app. It is an explainable algorithmic system that can answer what move is recommended, why it is recommended, and how risky the current state is while preserving a reusable engine that can later power tests, bots, Monte Carlo analysis, or a different UI front-end.

Why Strategy Analysis Matters

- **Best Move**converts

GAME ACTIONS

Hit Stand Double Split

Single-split support with multi-hand resolution.

ENGINE SHAPE

OOP Stateful UI-agnostic

Domain objects are isolated from rendering concerns.

ANALYTICS

Session Round Advice

Tracks outcome quality and recommendation adherence.

PERSIS

Sav Loa Exp

Snapsh restore bankrol and ag metrics

Key technical strengths. The project is strongest where portfolio apps usually stop: algorithm explainability, explicit payout accounting, multi-hand split state, live UI-state synchronization, and a premium JavaFX front-end with reus animation helpers.

basic-strategy rules into explainable decisions for the current hand.

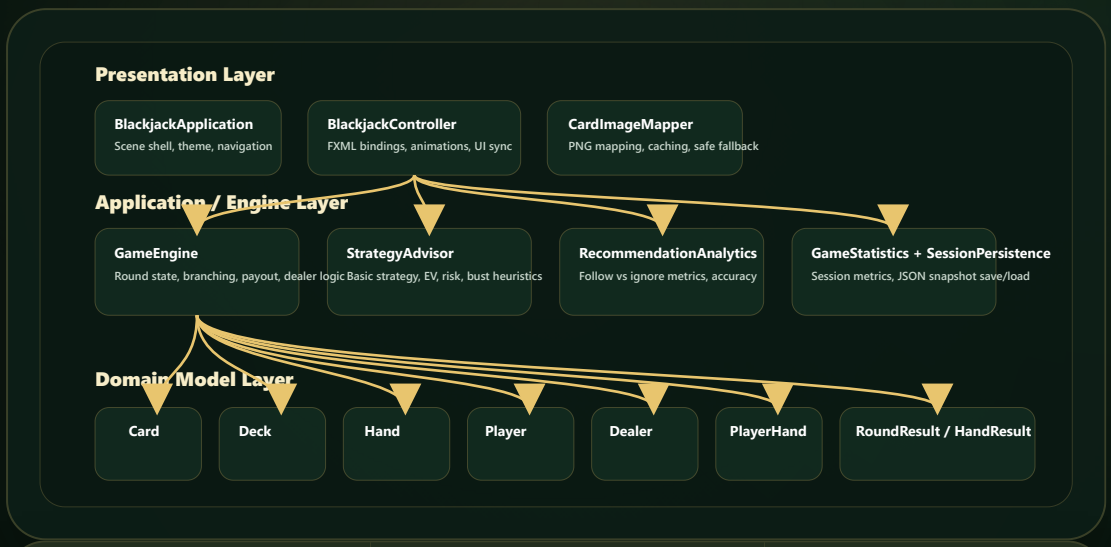
- **Expected EV**provides a normalized heuristic estimate of long-term hand quality.
- **Bust Risk**quantifies immediate risk when drawing one additional card.
- **Dealer Bust Chance**exposes the pressure created by a weak dealer up-card.
- **Recommendation Analytics**measures whether following advice correlates with better outcomes across the session.

Engineering Assumptions

- A fresh 52-card deck is rebuilt and shuffled at the beginning of every
- The dealer stands on all 17 values, including soft 17, because the draw is `value < 17`.
- Natural blackjack is detected immediately after the opening deal.
- Natural blackjack currently pays even money in the engine: payout = `bet`, not 3:2.
- The current implementation allows one split per round, which simplifi multi-hand sequencing and UI management.

03. CORE OOP ARCHITECTURE

Class Responsibilities And Layering

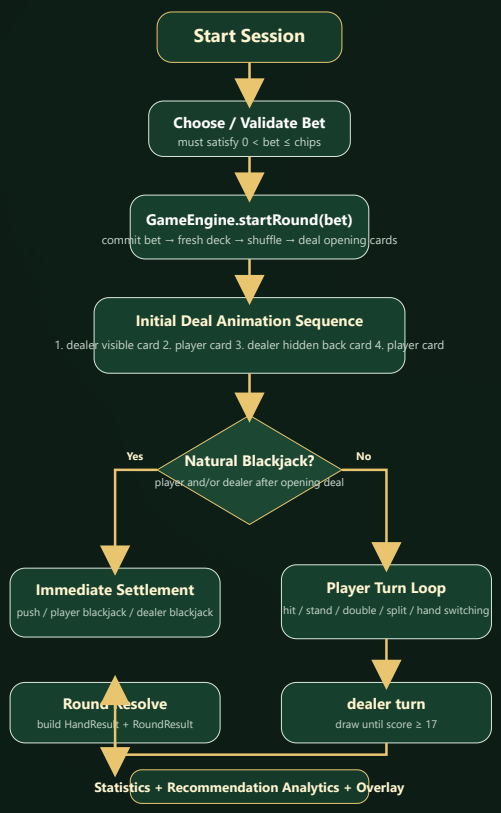


CLASS	PRIMARY RESPONSIBILITY	KEY DESIGN NOTES
Card	Immutable suit/rank object with nominal blackjack value and ace detection.	Separates symbolic identity from hand logic. Ace flexibility is handled in Hand, not here.
Deck	Builds a standard 52-card deck, shuffles it, and deals in sequence.	Reset/shuffle is round-scoped. The current engine uses a fresh deck every round rather than a reusable shoe.
Hand	Tracks cards, computes best blackjack value, soft state, blackjack, bust, and split eligibility.	The ace downgrade loop is the core of correct value evaluation.
Player	Owns bankroll and enforces valid bet commitment / payout return.	Bankroll mutation is centralized here, which prevents duplicate UI-side chip changes.
Dealer	Owns dealer hand and dealer draw rule.	Dealer policy is deliberately tiny: draw while value < 17.
GameEngine	Coordinates the full round lifecycle: bet commit, initial	This is the main orchestration boundary.

CLASS	PRIMARY RESPONSIBILITY	KEY DESIGN NOTES
	deal, action branching, dealer turn, settlement, statistics.	UI code reads from it but does not own rule transitions.
`RecommendationAnalytics`	Tracks whether users follow or ignore recommendations and what outcomes those choices lead to.	Decisions are recorded per hand and attached to final `HandResult` objects after Settlement.
`StrategyAdvisor`	Produces best move, reason, confidence, EV, bust risk, and dealer bust chance.	It is rule-based and explainable, with fast heuristics rather than Monte Carlo search.
`RoundResult`	Captures the resolved state of a round: outcome, chip delta, total bet, total payout, and per-hand details.	Acts as the handoff object to UI overlays, statistics, and analytics.
`SessionStatistics`	Implemented as `GameStatistics` plus snapshot DTOs for persistence and export.	Tracks wins, losses, pushes, blackjacks, split usage, double-down usage, win streaks, and total P/L.

04. GAME FLOW ALGORITHM

Round Lifecycle And Branching



Step-by-Step Lifecycle

1. **Bankroll + bet validation.** The player must have enough chips to commit the opening stake.
2. **Fresh deck initialization.** The engine creates a new deck and shuffles it every round.
3. **Opening hand creation.** The engine builds one `PlayerHand` and deals two player cards and two dealer cards.
4. **Natural blackjack check.** Before normal play begins, the engine resolves immediate blackjack outcomes.
5. **Player decision loop.** The active hand can hit, stand, double down, or split, subject to availability.
6. **Multi-hand sequencing.** Split hands are queued sequentially, and the active index advances to the next open hand.
7. **Dealer turn.** If at least one player hand survives, the dealer draws until the policy threshold is met.
8. **Settlement.** Each hand becomes a `HandResult`; the round becomes a `RoundResult`.

9. **Session update.** Statistics, recommendation analytics, overlays, chips, and labels refresh from the resolved result object.

Initial dealing split between engine and UI. The engine mutates authoritative game state immediately when a round begins. The UI then plays the opening sequence against that already-resolved state, which keeps animations deterministic without moving rule logic into JavaFX.

PAYOUT SETTLEMENT CONTRACT

```
B = committed bet for
a hand
P = payout returned to
bankroll
Δchips = P - B

Normal win:        P =
2B,  Δchips = +B
Push:              P =
B,   Δchips = 0
Loss / bust:       P =
0,   Δchips = -B
Natural blackjack:
current implementation
also uses P = 2B
```

05. STRATEGY RECOMMENDATION ALGORITHM

Decision Engine, Heuristics, And Analysis Fields

Recommendation Pipeline

1. Read the currently active hand and the dealer's visible up-card.
2. Map the dealer card into a strategy value where face cards become 10 and ace becomes 11.
3. If split is available and the hand is a pair, evaluate split rules first.
4. If not split, branch into `soft total` or `hard total` strategy tables.
5. Return the action plus reason string.
6. Derive analysis metadata: confidence, win tendency, risk level, EV, bust risk, dealer bust chance.

Why split rules come first. Pair splitting is not merely a total-based decision. It restructures the game tree into two separate betting branches, so it must be evaluated before soft or hard total logic.

PAIR SPLIT RULES	
A,A / 8,8	Always split
9,9	Split vs dealer 2-6, 8, 9

SOFT TOTAL RULES	
Soft 20	Stand
Soft 19	Double vs 6, else stand

HARD TOTAL RULES	
17+	Stand
13-16	Stand vs dealer 2-6, else hit
12	Stand vs 4-6, else hit

PAIR SPLIT RULES	
7,7	Split vs dealer 2-7
6,6	Split vs dealer 2-6
4,4	Split vs dealer 5-6
2,2 / 3,3	Split vs dealer 2-7
5,5 / 10-value pair	Do not split

SOFT TOTAL RULES	
Soft 18	Double vs 3-6, stand vs 2/7/8, hit vs 9/10/A
Soft 17	Double/hit vs 3-6, else hit
Soft 15-16	Double/hit vs 4-6, else hit
Soft 13-14	Double/hit vs 5-6, else hit

HARD TOTAL RULES	
11	Double vs non-ace, else hit
10	Double vs 2-9, else hit
9	Double vs 3-6, else hit
8 or less	Hit

CONFIDENCE

Confidence is highest for canonical strategy states such as splitting aces or eights, doubling hard 11 against a non-ace, or hitting hard totals that are clearly below 12. Edge cases degrade to medium confidence.

RISK & TENDENCIES

Risk rises for double down, split branches, and hard hit states near bust thresholds. Win tendency is based on hand softness, total strength, and dealer weakness or strength.

05 / 08

06. MATHEMATICAL LOGIC

Formulas, Heuristics, And Normalized Metrics

ACE-AWARE HAND VALUE

```
total = Σ
nominalCardValue
aces = count(Ace)

while total > 21 and
aces > 0:
    total = total - 10
    aces = aces - 1

bestValue = total
softHand = at least one
Ace remains valued as 11
```

EXPECTED VALUE HEURISTIC

```
EV =
    softBias
  + dealerWeaknessBias
  + totalStrengthBias
  + actionBonus
  - actionBustPenalty
  + 0.42 × DealerBustChance

Clamp(EV, -0.65, +0.65)
```

This is a normalized engineering heuristic, not exact combinatorial EV. It is intentionally fast, explainable, and stable under frequent UI refresh.

BUST RISK

```
BustRisk(hand) =
    Σ [ w(r) × I(
bust(hand + r) ) ] / Σ
w(r)

where:
    w(2..9) = 4
    w(10/J/Q/K bucket) =
16
    w(A) = 4
```

The strategy engine simulates a one-card draw against a weighted abstract rank distribution, which is fast enough for every UI refresh.

WIN RATE, SESSION PROFIT, RECOMMENDATION ACCURACY

```
ResolvedHands = wins + losses + pushes
WinRate = wins / ResolvedHands

SessionProfit = Σ roundChipDelta

FollowRate =
totalRecommendationsFollowed /
totalRecommendationsShown

RecommendationAccuracy =
    (followedWins + followedPushes) /
resolvedFollowedCount
```

PAYOUT AND BANKROLL INTEGRITY

```
On round start:
    player.commitBet(B)

On split / double:
    player.commitBet(additionalStake)
```

DEALER BUST CHANCE

```
DealerBustChance(upCard)
= lookup(upCard)

2 → 0.35   3 → 0.37   4
→ 0.40   5 → 0.42   6 →
0.42
7 → 0.26   8 → 0.24   9
→ 0.23   10 → 0.21   A →
0.17
```

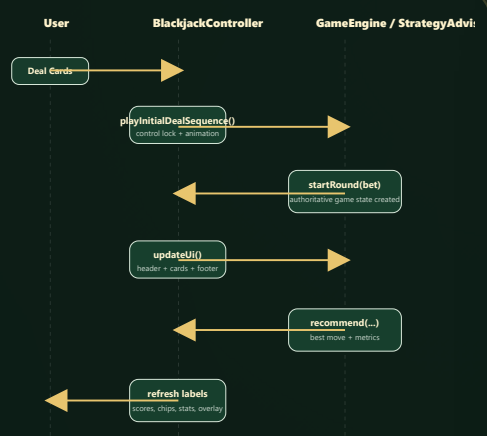
```
At settlement:

player.receivePayout(totalPayoutAmount)

ChipDelta = totalPayoutAmount -
totalCommittedBet
```

The engine performs payout exactly once during `finishRound(...)`, which prevents duplicate bankroll updates from UI actions.

METRIC	INTERPRETATION	ENGINEERING TRADEOFF
Expected EV	Normalized directional estimate of long-term value for the recommended action.	Fast enough for live UI updates, but intentionally approximate.
Bust Risk	Probability that drawing one more card immediately busts the current hand.	Uses weighted ranks, not exact remaining-deck composition.
Dealer Bust Chance	Precomputed weakness estimate derived from the visible dealer up-card.	Near constant-time and stable; ideal for UI explanation.
Confidence	How strongly the decision matches canonical strategy patterns.	Categorical, human-readable, intentionally not statistical confidence.

07. UI-STATE SYNCHRONIZATION

Keeping JavaFX Presentation Aligned With Engine State



Synchronization Strategy

- **Engine-first state.** Rules update first; animations render the already-valid state.
- **Single refresh hub.** `updateUi()` coordinates headers, cards, recommendation panel, footer metrics, controls, and round-end effects.
- **Animation gating.** The controller uses explicit flags such as `animationInProgress` and `initialDealAnimationInProgress` to keep buttons disabled during transitions.
- **Split-hand awareness.** The active hand index drives highlighting, score display, and recommendation context.
- **Resolved-state handoff.** Once a round is over, recommendation values switch from live analysis to a safe “round complete” state, while overlays and settlement chip animations take over.

UI CONCERN	UPDATE TRIGGER	SOURCE
card dealing animation	opening deal, hit, dealer draw, dealer reveal	Controller engine-ov

UI CONCERN	UPDATE TRIGGER	SOURCE
chip balance	Bet commit, clear bet, settlement, reset, session load	`Player.c` animatior
hand score label	Initial deal finish, hit completion, dealer reveal/draw	`Hand.get`
recommendation panel	Initial deal, every hit, split, hand switch, double, dealer reveal, round resolve, session load	`Strategy`
session save/load	Utility buttons	Snapshot serializer/

07 / 08

08. PERFORMANCE & OPTIMIZATION

Optimization Opportunities And Rendering Efficiency

CURRENT PERFORMANCE CHARACTERISTICS

- Fresh deck + shuffle per round keeps state simple and deterministic.
- PNG card images are cached in memory after first load.
- Dealer bust probabilities are constant-time lookups.
- Bust risk runs against a ten-rank weighted distribution, not full deck permutations.

OPTIMIZATION OPPORTUNITIES

- `(hand signature, dealer up-card, action)` by `(hand signature, dealer up-card, action)`.
- Upgrade from per-round deck rebuilds to a multi-deck shoe for casino realism.
- Add split recursion caching when multiple re-splits are introduced.
- Batch some JavaFX node refreshes when large split layouts are active.

RENDERING OPTIMIZATIONS

- Use a dedicated animation layer so transient cards and chips do not relayout the entire scene.
- Preserve consistent card dimensions to avoid layout thrash during dealing.
- Keep card images rasterized and cached to avoid runtime SVG decoding cost.
- Separate stable felt/background layers from dynamic overlays.

09. FUTURE UPGRADE ROADMAP

Strategic Expansion Paths

ROADMAP ITEM	TECHNICAL VALUE	IMPLEMENTATION DIRECTION
Monte Carlo simulation mode	Exact probability surfaces and stronger EV estimates.	Add simulation workers behind the same recommendation interface.
reinforcement learning strategy	Enables comparative analysis against basic strategy.	Introduce pluggable policy engines with offline training data.
multiplayer support	Expands the engine from one player to seat-based table state.	Generalize `PlayerHand` ownership and turn sequencing.

ROADMAP ITEM	TECHNICAL VALUE	IMPLEMENTATION DIRECTION
online leaderboard	Adds persistence, ranking, and product-like engagement.	Persist snapshots to a backend service or local database.
probability heatmap	Makes recommendation confidence and EV visually explorable.	Render matrix overlays from strategy engine outputs.
mobile version	Validates engine portability beyond desktop JavaFX.	Reuse engine and analytics layers behind a mobile UI shell.

10. FINAL ENGINEERING SUMMARY
Why This Reads As A Real Engineering Portfolio Piece

This project demonstrates a strong full-stack engineering profile because it treats a seemingly simple game as a serious systems problem. The codebase combines a clean OOP domain model, explicit state transitions, rule-based recommendation logic, analytics instrumentation, persistence, and a premium JavaFX desktop front-end.

The most portfolio-worthy characteristic is not that the application can deal cards. It is that the application can **justify decisions**, **measure behavioral outcomes**, **animate state without corrupting logic**, and **persist a meaningful session model**. Those qualities make the project suitable for GitHub showcase material, system design discussion, and interview walkthroughs that emphasize architecture, correctness, and product thinking.

ARCHITECTURE

Reusable Engine

Rules stay independent from JavaFX rendering.

ANALYSIS

Explainable Strategy

Every recommendation is paired with rationale and metrics.

PRESENTATION

Premium Desktop UX

Casino-style visuals, animations, overlays, and session tools.

Technical note: this document reflects the current implementation faithfully, including even-money blackjack payout, one-split-per-round support, and heuristic EV/risk models designed for fast UI refresh.